

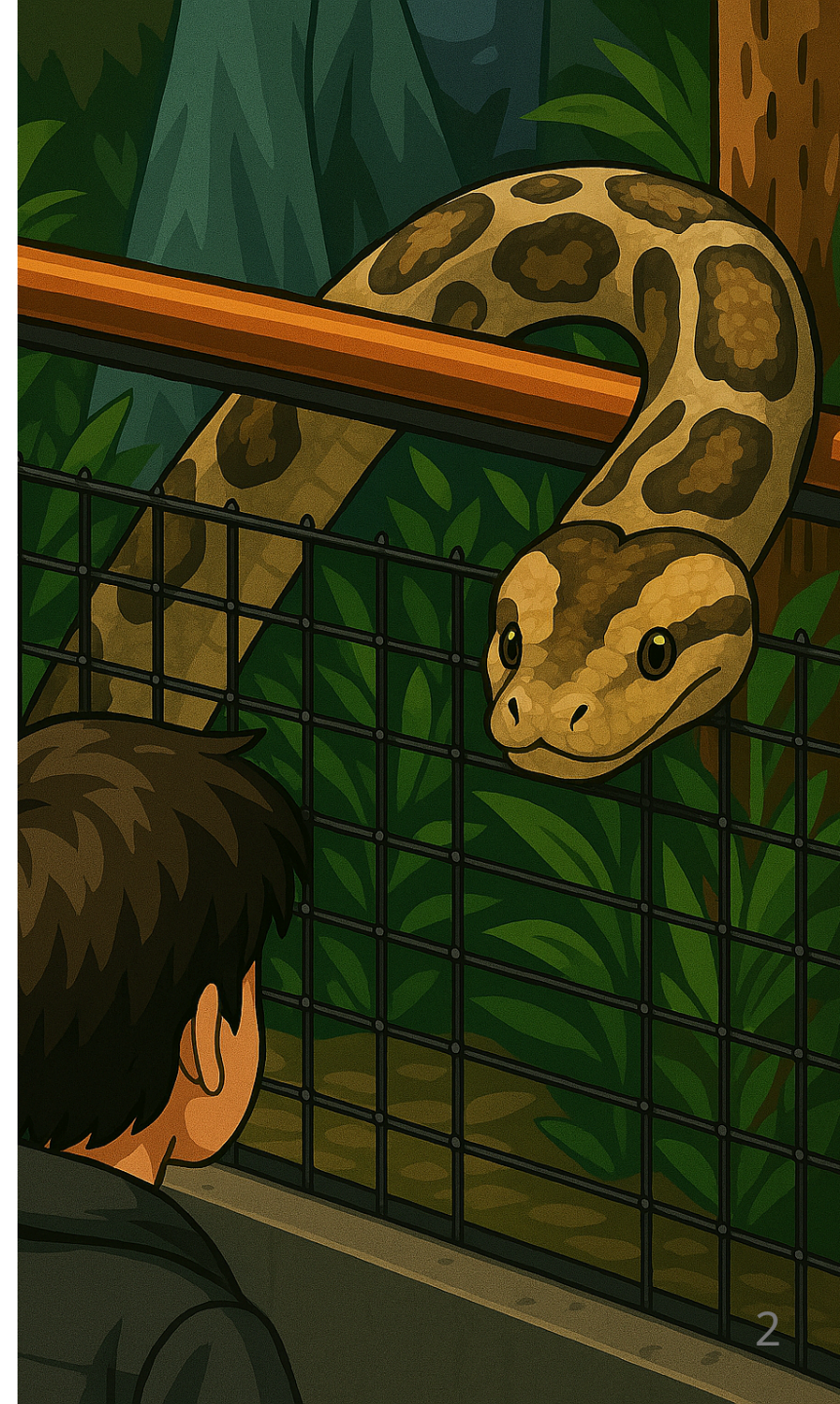
Introduction au langage Python

BTS CIEL



Sommaire

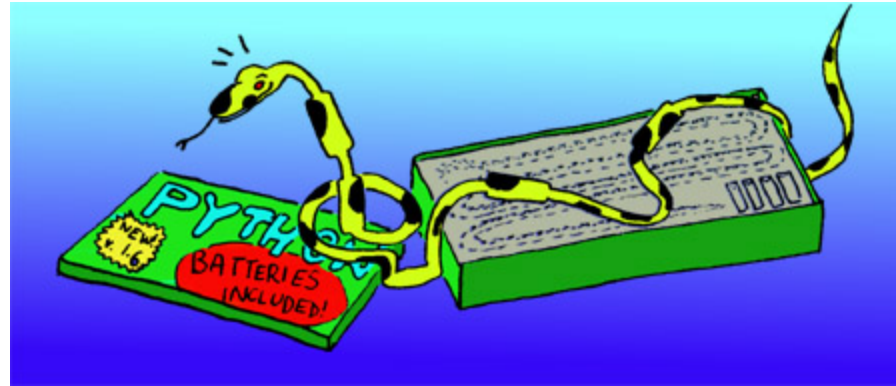
- Pourquoi apprendre Python ?
- Programmation structurée
 - Instructions, variables et blocs
 - Séquence
 - Sélection
 - Itération
- Liste
- Chaîne de caractères
- Fonctions
- Autres capacités du langage



Pourquoi apprendre le Python ?

- **GPL (General-purpose Programming Language) :**
 - Web : Django, Flask, FastAPI
 - IA et data : Numpy, Panda, PyTorch
 - Administration Système : cross-platform, pré-installé sur Linux, interprété
 - Programmation embarqué : MicroPython, CircuitPython, MicroBit
- Simplicité d'apprentissage
 - Syntaxe allégée pour se concentrer sur la logique
 - Peu de "cérémonie" / pas de compilation
- Populaire dans les milieux professionnel et scolaire :
 - N°1 sur l'Index [Tiobe \(27% des parts\)](#) et sur [PYPL \(31%\)](#)
 - En France : langage de la recherche et l'éducation nationale

Pourquoi apprendre le Python ?



Python fourni environ ~300 modules :

- Fichiers / OS : `os` , `pathlib` , `shutil` , `tempfile` , `subprocess`
- Réseau : `http` , `socket` , `asyncio` , `ssl`
- Données : `csv` , `json` , `sqlite3` , `pickle`
- Math : `math` , `random` , `decimal` , `fractions` , `statistics`
- Web / formats : `html` , `xml` , `email` , `cgi`

Pourquoi apprendre le Python ?

```
from datetime import datetime

import http.server, socketserver, json, random, logging

logging.basicConfig(level=logging.INFO)

class Handler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
        data = {"time": datetime.now().isoformat(), "value": random.randint(1, 100)}
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(data).encode())

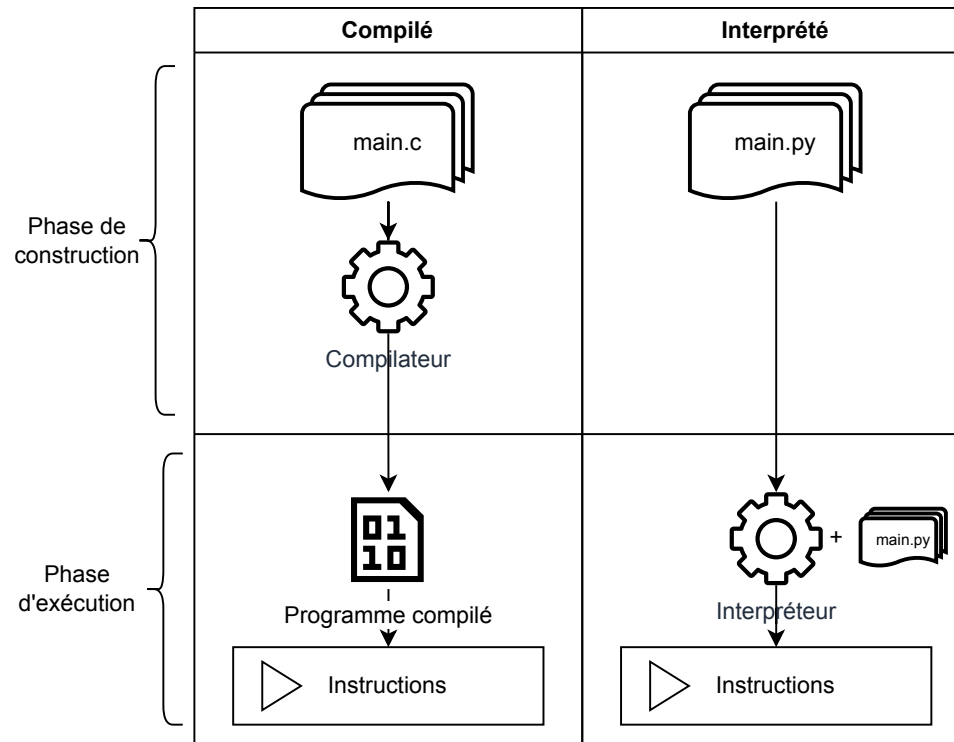
with socketserver.TCPServer(("", 8000), Handler) as httpd:
    logging.info(f"Serving on port 8000")
    httpd.serve_forever()
```



Permettre de construire un **logiciel** complet sans ajouter de bibliothèques tiers.

Langage interprété (CPython)

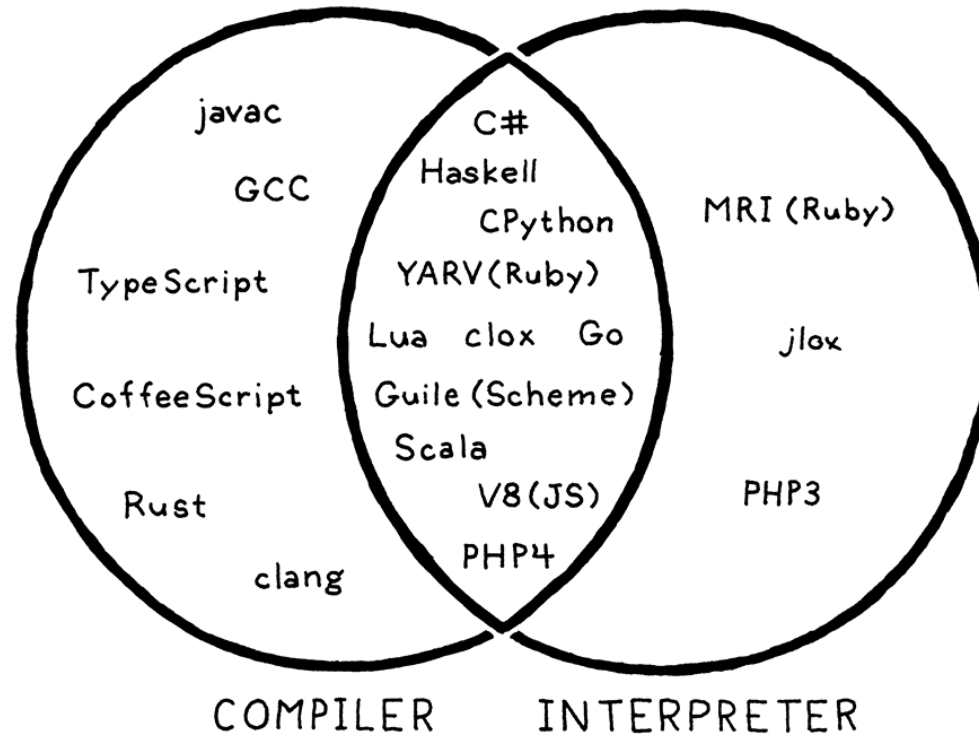
Interprété VS compilé



i Python est un langage interprété.

Langage interprété

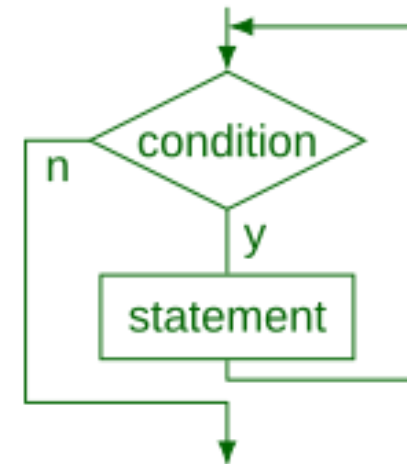
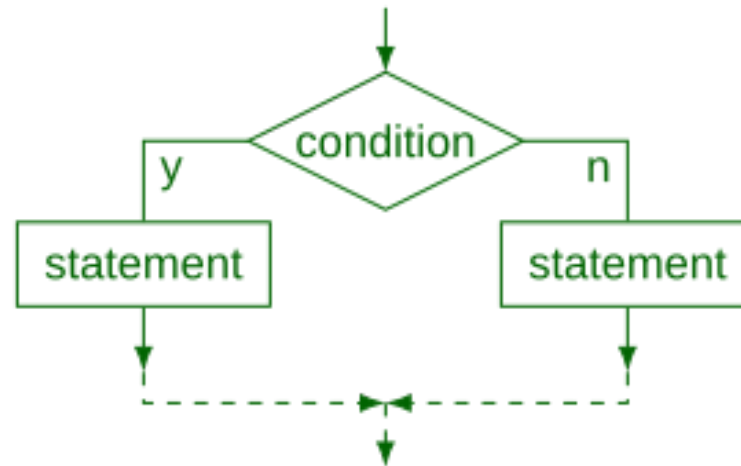
La réalité



Programmation structurée

La programmation structurée définit les primitives suivantes

- ▼ La séquence
- ↻ La sélection
- ↺ L'itération



▼ Séquence - point d'entrée du programme

En Python il n'est pas nécessaire d'écrire le code dans une fonction principale (`main`).

Cependant vous **pouvez** :

- Préciser l'interpréteur à utiliser pour votre script (shebang) en positionnant l'instruction suivante en début de fichier :

```
#!/usr/bin/env python3
```

- Différencier l'usage que l'on fait de votre script (execution ou import) :

```
if __name__ == "__main__":  
    print("Ce code sera exécuté uniquement si le script est exécuté.")  
  
print("Ce code sera exécuté dans tout les cas.")
```

▼ Séquence - point d'entrée du programme

Si l'on veut copier la structure d'un programme C :

```
#!/usr/bin/env python3

def main():
    print("Hello World!")

if __name__ == "__main__":
    main()
```

Séquence - variable

```
ma_variable = 10 # integer  
mon_autre_variable = "du texte" # string  
mon_bool = True # boolean
```

▼ Séquence - typage

Types natifs du langage :

- Numériques `int`, `float`, `complex`
- Séquences `list`, `tuple`, `range`, `str`, `bytes`
- Dictionnaires (mapping) `dict`
- Classes, instances et exceptions

i Natif signifie qu'ils sont inclus dans l'interpréteur Python.

i D'autres types peuvent être fournis par des bibliothèques (Numpy, Panda, etc.)

▼ Séquence - typage

Python utilise un typage dit **dynamique** et **strique**.

```
points = 3.2 # points est du type float
print("Tu as " + points + " points !") # Génère une erreur de typage

points = int(points) # points est maintenant du type int (entier), sa valeur est arrondie à l'unité inférieure (ici 3)
print("Tu as " + points + " points !") # Génère une erreur de typage

print("Tu as " + str(points) + " points !") # Plus d'erreur de typage, affiche 'Tu as 3 points !'
```

Le C utilise un typage dit **statique** et **faible**.

```
float points = 3.2; // Typage statique : points est un float
points = (int) points; // Erreur points est de type float elle ne peut pas recevoir un int

int pts = (int) points; // Conversion explicite : float → int
printf("Tu as %s points !\n", pts); // Typage faible : conversion implicite
```

⚠ En Python l'erreur sera levée lors de l'exécution du programme

Séquence - typage

Définition

Typage Dynamique / statique Est-ce qu'une variable de type **A** peut contenir une valeur de type **B** sans être redéfinie.

Typage faible / fort Est-ce qu'une variable de type **A** peut-être utilisée en tant que type **B** (sans cast explicite)

▼ Séquence - opérations mathématiques

Python défini trois types numériques : `int` , `float` , `complexe`

La liste des opérations de bases possibles sur ces types :

Opération	Définition
<code>x + y</code>	somme de x et y
<code>x - y</code>	différence de x et y
<code>x * y</code>	produit de x et y
<code>x / y</code>	quotient de x et y
<code>x // y</code>	quotient entier de x et y
<code>x % y</code>	reste de x / y
<code>-x</code>	l'opposé de x

▼ Séquence - instruction et bloc

En Python il n'est pas nécessaire de terminer une instruction par un `;`.

Les blocs sont représentés en utilisant le caractère `:` suivi de l'indentation.

(en lieu et place de `{}`) :

```
x = int(input("Entrez un nombre : "))  
  
if x > 2:  
    print("X est supérieur à 2.")  
else:  
    print("X n'est pas supérieur à 2.")
```

▼ Séquence - appel de fonction

L'appel de fonction se fait en utilisant le **nom de la fonction**, suivi de la **liste des arguments** entre parenthèses :

```
print("Hello world !")  
le_max = max(10, 15)
```

Fonctions équivalentes à `printf` et `scanf` :

```
x = int(input("Entrez un nombre :"))  
print(f'Vous avez saisi le nombre {x}')
```

Sélection - if / elif / else

La sélection permet de modifier le "chemin" emprunté par le programme, en fonction de la valeur d'une **expression booléenne**.

```
ma_variable = 10
if ma_variable == 11:
    print("foo")
elif ma_variable == 12:
    print("or")
else:
    print("bar")

# Affichera : "bar"
```

Sélection - if / elif / else

L'instruction `if` doit être suivie d'un **bloc** de code. Les règles d'indentation sont ici très importante.

`if` peut-être seul ou accompagné de :

- un ou plusieurs `elif` ;
- d'un seul `else` .

Les instructions `if` peuvent être imbriquées pour former des instructions de sélection plus complexes.

Sélection - if / elif / else

```
a = 10
if a >= 10:
    if a <= 50:
        print("Inférieur ou égal à 50.")
    elif a <= 60:
        print("Inférieur ou égal à 60 mais supérieur à 50.")
    elif a <= 80:
        print("Inférieur ou égal à 80 mais supérieur à 60.")
    else:
        print("Supérieur à 80.")
else:
    print("Inférieur à 10.")
```

Sélection - opérations de comparaison

Opération	Définition
<	strictement inférieur
<=	inférieur ou égal
>	strictement supérieur
>=	supérieur ou égal
==	égal
!=	différent
is	identité d'objet
is not	contraire de l'identité

Sélection - véracité des types

Considérés comme étant **faux** :

- Les constantes `None` et `False`
- zéro de tout type numérique : `0`, `0.0`, `0j`
- les chaînes et collections vides : `''`, `()`, `[]`, `{}`

Tout le reste est considéré comme étant **vrai**.

Sélection - opérations booléennes

Opération	Définition
<code>x or y</code>	si x est vrai alors x, sinon y
<code>x and y</code>	si x est faux, alors x sinon y
<code>not x</code>	si x est faux, alors <code>True</code> sinon <code>False</code>

Sélection - pattern matching

Equivalent du `switch` en langage C :

```
match status:
    case 400:
        print("Bad request")
    case 404:
        print("Not found")
    case 418:
        print("I'm a teapot")
    # Cas par défaut
    case _:
        print("Code erreur inconnu")
```

Sélection - pattern matching

Permet aussi de faire des opérations plus élaborées :

```
match point:
    case (0, 0):
        print("Origin")
    case (0, y):
        print(f"Y={y}")
    case (x, 0):
        print(f"X={x}")
    case (x, y):
        print(f"X={x}, Y={y}")
```

Itération - while loop

```
i = 0

while i < 10:
    print(i)
    i += 1

# Résultat: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

animals = ["chien", "chat", "souris"]
i = 0

while i < 3:
    print(animals[i])
    i += 1

# Résultat: "chien", "chat", "souris"
```

Itération - for loop

```
animals = ["chien", "chat", "souris"]  
  
for animal in animals:  
    print(animal)  
  
# Résultat: "chien", "chat", "souris"
```


Itération - fonction range

La fonction `range` permet de générer une **suite arithmétique** entre deux bornes.

Elle est souvent utilisée avec l'instruction `for` pour itérer un nombre de fois.

La borne supérieure est **exclue**.

```
ma_liste = list(range(5, 10))  
# [5, 6, 7, 8, 9]  
  
for e in ma_liste:  
    print(e)  
  
# Résultat: 5, 6, 7, 8, 9  
  
for i in range(0, 10):  
    print(i)  
  
# Résultat: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

Liste

La structure qui se rapproche le plus d'un tableau (langage C) est la liste :

```
fruits = ['orange', 'pomme', 'poire', 'banane', 'kiwi']  
fruits[0] # orange
```

à l'inverse d'un tableau, une liste Python peut contenir des éléments de différents types :

```
fruits_et_code = ['orange', 22, 'poire', 40, 'kiwi', 42]
```

La taille d'une liste n'est pas fixe (on peut ajouter des éléments) :

```
fruits.append("mangue")
```

Chaîne de caractères (string)

à la différence du langage C, Python propose un type natif `string` permettant de manipuler les chaînes de caractères :

```
un_string = "LEIC STB"
un_deuxieme = 'BTS SNIR'

# un string se manipule comme une séquence
resultat = ''.join(reversed(un_string)) + f' anciennement {un_deuxieme[3:]}'

print(resultat) # BTS CIEL anciennement SNIR.
```

Chaîne de caractères (string)

Cet exemple nous montre que :

- Python ne différencie pas une chaîne de caractère et un caractère simple (`'` peut-être utilisé à la place de `""`)
- Il est possible de concaténer deux `string` via l'opérateur `+`
- Les `string` se manipule comme des séquences (`reversed` et `[3:]`)

Fonctions en Python

Principe

Le principe de fonction en Python et C sont équivalent :

- Une fonction est un **bloque de code** (nommé) **qui est exécuté lorsqu'il est appelé**
- Une fonction peut avoir des **arguments** et retourner un **résultat**
- Une fonction sert à éviter la **duplication de code**.

Fonctions en Python

Définition et appel de fonction

Le mot clé `def` permet de définir une fonction

```
def affiche_un_texte():  
    print("Ma super fonction")  
  
def somme(x, y):  
    # On peut appeller une autre fonction  
    affiche_un_texte()  
  
    # Retourner un résultat  
    return x + y  
  
print("Résultat = " + str(somme(10, 12)))  
  
# Ma super fonction  
# Résultat = 22
```

Encore beaucoup de chose ...

35 mots clés contre 32 en C (19 si on retire les mots clés en lien avec le typage) :

False	None	True	and	as	assert	async	await
break	class	continue	def	del	elif	else	except
finally	for	from	global	if	import	in	is
lambda	nonlocal	not	or	pass	raise	return	try
while	with	yield					

Certains mot clés permettent d'utiliser des mécanismes très avancés : `raise` , `class` , `yield` , `with` .

Langage multi-paradigme

En programmation, un paradigme correspond à une manière de formuler la réponse à un problème.

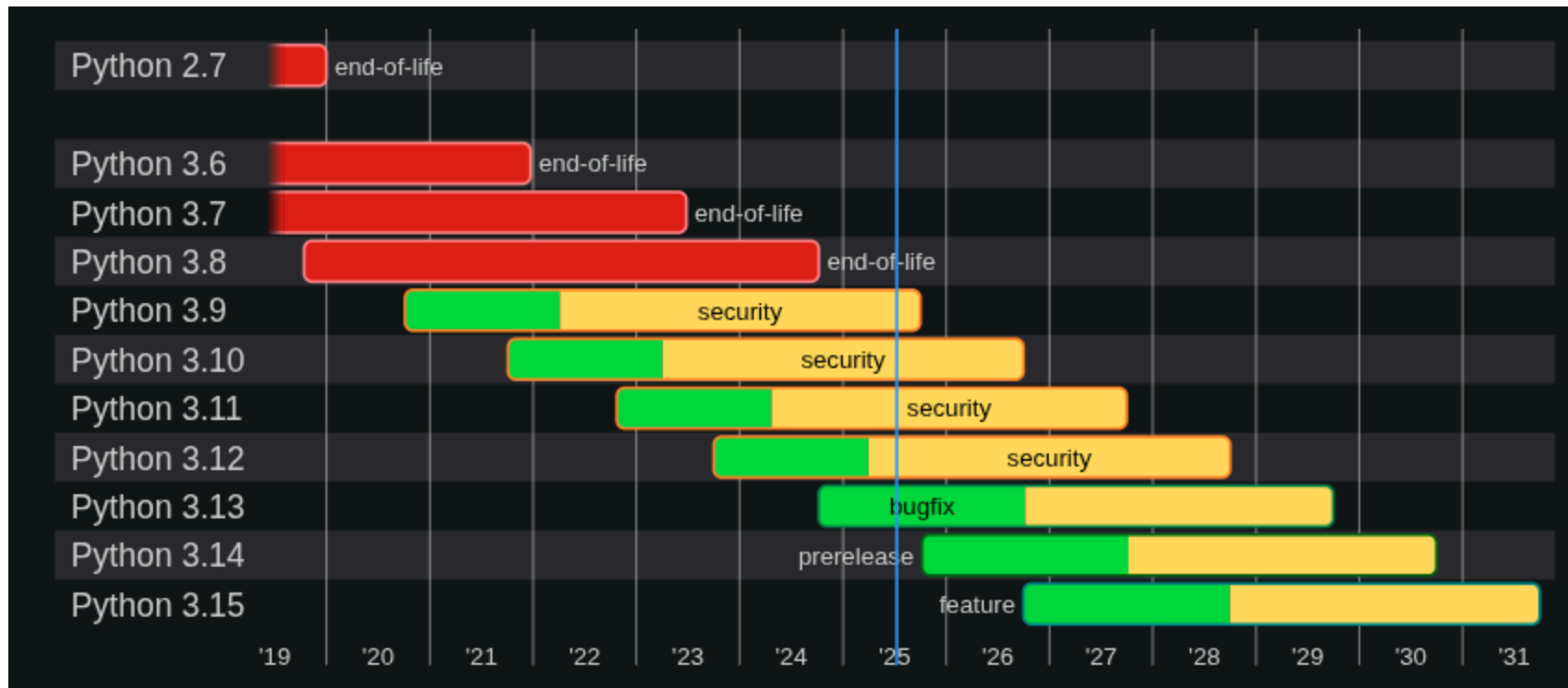
- Langage C : programmation impérative (structurée et procédurale)
- Python : programmation impérative, **fonctionnelle** et **orientée objet**.

Python vous offre plusieurs manières de décrire un programme : à vous de choisir celle qui convient le mieux à votre besoin.

Env de développement - versions de Python

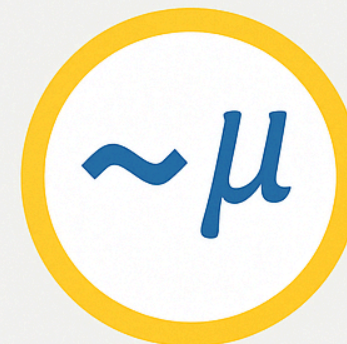
Pour connaître la version de python (3) installée

```
python3 --version
```



Env - éditeurs

- Visual Studio Code
 - Python
 - Python Debugger
- PyCharm
- Qt Creator
- Mu et Micro:bit Python Editor



Suite du cours

- Structure avancées (liste, dictionnaire, ...)
- Gestion des exceptions
- Interaction avec l'environnement (lecture de fichiers, programmation réseau, IHM, ...)
- **Programmation Orienté Objet**



Liens et sources

- <https://docs.python.org/3/tutorial>
- <https://craftinginterpreters.com/a-map-of-the-territory.html>
- https://en.wikipedia.org/wiki/Structured_programming